

Project Update Report

A. Project Scope Update

This project aims to fine-tune a large language model (Mistral-7B-Instruct) using LoRA (Low-Rank Adaptation) for emotion classification. The scope has expanded beyond initial fine-tuning to include dataset balancing, prompt engineering, and performance evaluation using key classification metrics such as Accuracy, Macro F1 Score, and Confusion Matrix.

Model training is conducted on Google Colab using available GPU resources (T4 or H100). Multiple experiments are performed with different hyperparameter configurations (e.g., batch size, gradient accumulation) to optimize performance under computational constraints.

B. Data sources

These datasets are in JSONL format and consists of instruction-based samples, including input text and corresponding emotional labels. It is processed using the Hugging Face Datasets library, with partial sourcing from Kaggle. In addition, I will also split the dataset into train, validation and test to fit those data into model building.

Data_processing includes the data cleaning and prompt transformation. In addition, this python code will divide datasets into several sentences correspond to each label. All labels are standardized into seven emotion categories: joy, sadness, anger, fear, surprise, disgust, and neutral. To mitigate class imbalance—particularly the dominance of the neutral class—data preprocessing also includes sampling strategies to improve label distribution.

Check_json will focus on fetching data and tokenizer check. This check is aimed to make sure dataset is well prepared and no missing value or malfunctional prompt. This was finished in local instead of Colab is to save computing resources on it.

The implementation leverages Hugging Face Transformers and PEFT libraries for loading pretrained models and performing efficient LoRA-based fine-tuning. This approach significantly reduces memory and computational requirements compared to full model training.

C. Issue / difficulties

The primary challenge is class imbalance, which negatively impacts model performance, especially the Macro F1 score. The overrepresentation of the neutral class leads to biased predictions.

Computational limitations also present a major constraint. Google Colab restricts access to high-performance GPUs (H100), while T4 GPUs provide limited capability. In some cases, training must rely on CPU resources, significantly slowing down experimentation and reducing feasible dataset size.

Additional issues include:

- Instability in training loss due to small batch sizes: Due to the restriction of the computing restriction, the batch size must be set to small. This will lead to training loss because the model will be impacted by samples.
- Slow evaluation caused by sequential inference: Compared to the parallel training , the evaluation of the model is sequential which is much slower; therefore, I attempt to minimize the token length and eval steps to mitigate the problem.